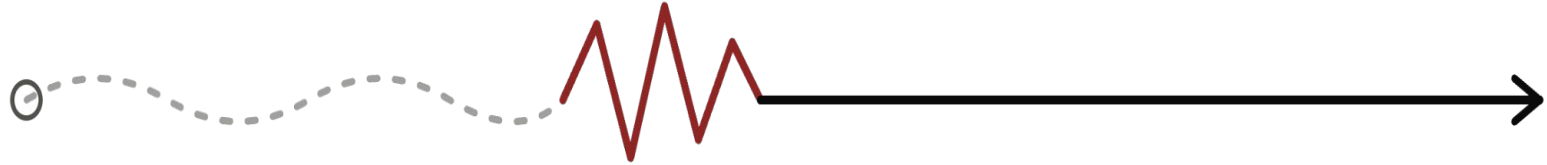


Vibes

Rigor



Product development at the speed of trust

AI SDLC

Lessons learned from the Field

SPEAKER

Matt Koch

SPEAKER

Matt Studer

HOSTED BY

Slalom

September 2026

Intros

SPEAKER

Matt Koch

Technology Advisor, former Director, Intelligence Engineering (AI/ML) at Slalom

Technology Advisor with years of experience helping teams and executives put AI to work on strategic initiatives, with expertise spanning machine learning, Agile, and cloud architecture.

Education

M.S. in Computer Science — University of Colorado Boulder

LinkedIn: <https://www.linkedin.com/in/ximsce/>

SPEAKER

Matt Studer

Senior Principal, Intelligence Engineering (AI/ML) at Slalom

Architects production-grade AI systems across industries — from enterprise generative AI platforms to LLM fine-tuning infrastructure at scale.

Education

B.S. Chemical Engineering & M.S. Applied Statistics — University of Alabama

LinkedIn: <https://www.linkedin.com/in/matthew-j-studer/>

Agenda & Context

What We'll Cover

- 01 Motivations
- 02 Technical Setup
- 03 Context Development
- 04 AI SDLC Workflow
 - Spec-Driven Development
 - The New Code Review
 - CI/CD
- 05 Future Considerations
 - Maintaining Quality
 - Working as a team
 - Scaling
 - Enterprise grade development

Assumptions

1. Technical Background

You have some technical background or interest

Comfortable in Git, at least one programming language, frontend & backend development

2. Vibe Coding Mindset

You've dabbled in Vibe coding and are interested in good vibes, not slop

Focusing on quality engineering, clean architecture, and practical AI SDLC practices

Technical Setup & Considerations

Covering today: OpenCode / Open Spec

Alternatives: Claude Code, Codex... / Spec-kit

Repo:

<https://github.com/ximsce/frame-clarity-analyzer>

References:

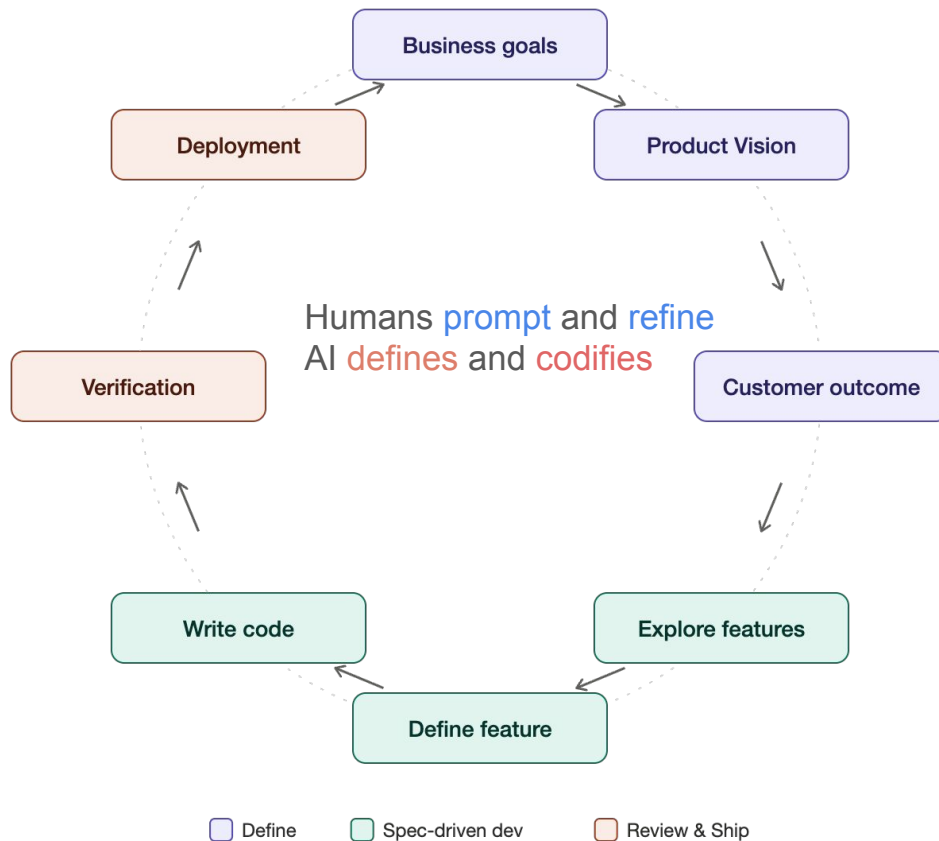
OpenCode: <https://opencode.ai/>

OpenSpec: <https://openspec.dev/>

[Explain-Diff](#) by Geoffrey Litt



AI SDLC



Before We Go Further: What LLMs Actually Are

Understanding the machine helps you use it well — and know when not to trust it.

What they are

Statistical pattern matchers trained on text

They predict the next most likely token — they **don't reason, plan, or understand** your codebase

Each prompt is a query into the latent space. Tight queries → tight results. Squishy queries → squishy results.

They have **no memory** between sessions unless you provide it

What that means for you

Context is everything — the AI can only be as good as what you give it

Specs matter because the AI will skip clarifying questions and just start building.

Tests are non-negotiable — confident-sounding code still requires independent verification

Application development shifts from writing code to **designing the validation system** around it

What Are Agent Harnesses?

The orchestration layer that turns an LLM into a working development partner.

What they are

Tools that wrap an LLM with file access, shell execution, and iterative prompting loops

They give the LLM **agency** — read code, run commands, see errors, try again

The loop: prompt → execute → observe → re-prompt

Extensible via tools, skills, and MCP — the harness can call external services, run custom scripts, and connect to APIs beyond just the filesystem

Why they matter

They close the feedback loop — the AI can see if its code actually works

Context injection — automatically feeds relevant files, structure, and conventions to the LLM

MCP (Model Context Protocol) lets the harness plug into databases, monitoring, Jira, Slack — anything with a server

Skills = reusable prompt recipes

Tools = actions the AI can invoke.

Together they make the harness composable.

Context Engineering

The same prompt produces different results depending on what the AI knows about your project.

WITHOUT Context

"Add a new API endpoint for photos"

- Express.js boilerplate with MongoDB
- Wrong framework, wrong DB, wrong patterns
- 2 hours fixing it

WITH Context

Same prompt

- Next.js handler with PostGres
- Follows existing repository pattern
- Matches naming conventions. Ready to use.

The Context Hierarchy

- 1 **Global Rules** — Your coding style — applies everywhere [per user](#)
- 2 **Project Context** — [AGENT.md](#) — architecture, conventions [per repo](#)
- 3 **Structural Context** — Codebase maps, dependency graphs, READMEs [per repo](#)
- 4 **Session Context** — The spec or task you're working on [per task](#)
- 5 **Prompt Context** — The specific instruction [per message](#)

Why Specs Matter

Underspecified prompts are the most common source of rework in AI-assisted development.



Scenario: You give the AI a vague request: "Build a notification system"

- The AI doesn't ask clarifying questions — it just starts building
- It picks a queue you don't use, creates an API that doesn't match your patterns
- You get working code that solves the wrong problem
- Now you're debugging AI assumptions instead of building features

Specs aren't overhead. They change AI from a guessing machine into a reliable collaborator.

Spec-Driven Development: The Approach

A spec defines what "done" looks like before the AI starts writing code.

The Approach

An approach, not a specific tool or package

Defines WHAT should happen: scenarios, behaviors, acceptance criteria

AI excels at implementing well-defined requirements. AI is terrible at guessing. The spec removes the guessing.

Works with ANY GenAI tool — it's how you structure work, not which IDE you use

OpenSpec — One Flexible Framework

Markdown-based, familiar to developers

Flexible — custom doc sections per project area

Not rigid (unlike Kiro's baked-in approach) — you control the spec flows

Tool-agnostic

Alternatives exist (GSD, SpecKit, custom) — the approach matters more than the package

Technical Setup & Considerations

Covering today: OpenCode / Open Spec

Alternatives: Claude Code, Codex... / Spec-kit

Repo:

<https://github.com/ximsce/frame-clarity-analyzer>

References:

OpenCode: <https://opencode.ai/>

OpenSpec: <https://openspec.dev/>

[Explain-Diff](#) by Geoffrey Litt



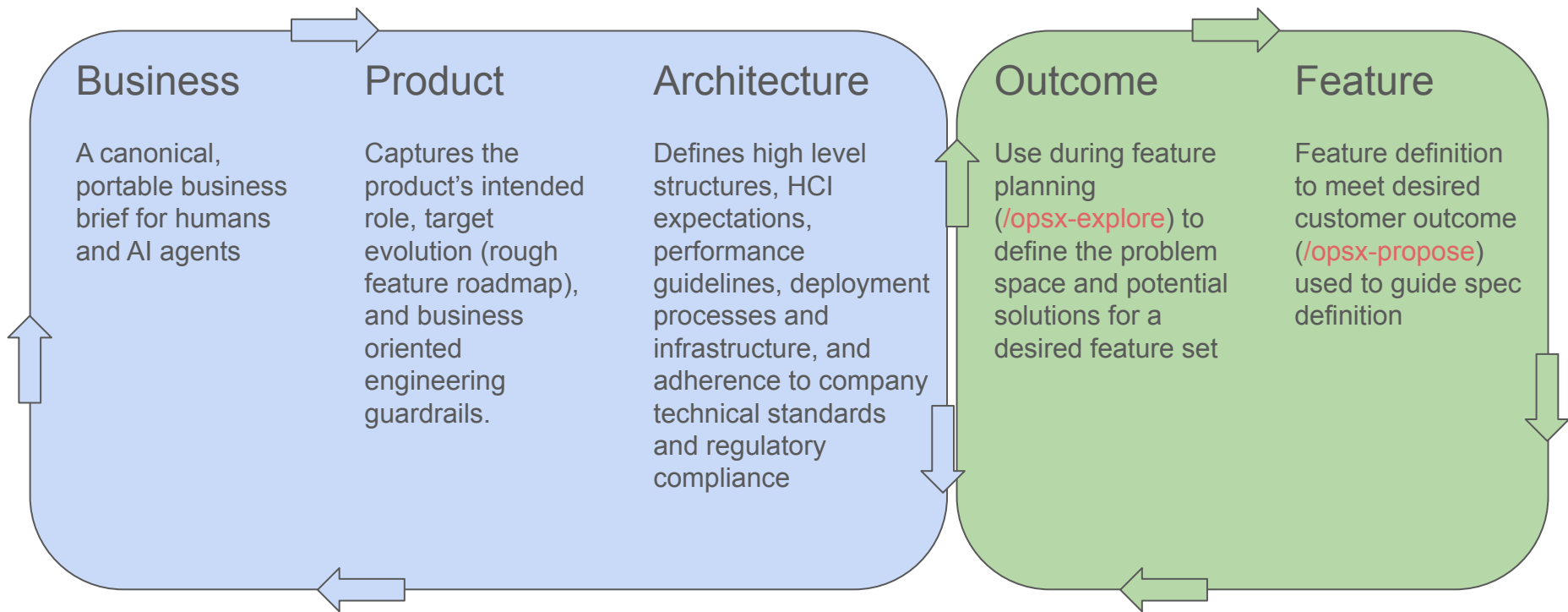
Introducing **Wedding Glamour**

Fake company and product based on a real-world need

Product Vision: Find the best still images from a memorable moment only captured in video



Context Development



Now we're ready to `/opsx-apply` our first change!

OpenSpec Workflow

`/opsx:explore` map the problem and understand the codebase

`/opsx:propose` draft proposal.md, specs/, design.md, [tasks.md](#)

`/opsx:apply` implement tasks from the specification

`/opsx:archive` archive completed changes

AI Product Considerations

PILLAR 01

Golden Data Set

- Curated ground-truth benchmark data
- Standardized evaluation metrics
- Continuous dataset refinement

PILLAR 02

LLM As Judge

- Automated output scoring
- Scalable regression testing
- Consistent criteria evaluation

PILLAR 03

Human in the Loop

- Expert validation & oversight
- Active feedback integration
- Edge case handling & safety

Verification: Building Trust and the New Code Review

The limits of spec-driven development

- A spec can be satisfied to the letter and still miss the intent
- AI-accelerated velocity outpaces line-by-line human review
- *The gap isn't effort — it's that "did we follow the spec" and "is this actually good" are different questions*

Build on existing best-in-class CI/CD tools

- SonarQube, linters, existing test suites — these are already parts of the SDLC toolbox, now **more important than ever**
- AI review adds a layer; it doesn't remove your quality gates

The new code review

- Agentic review in CI/CD — an AI reviewer runs before a human ever opens the PR
- `/explain-diff (/show-me)`— AI narrates *what* changed and *why*, so humans review intent, not syntax

Production outcome monitoring

- The spec was a hypothesis — production is the proof
- Close the loop by measuring the customer outcome you started with

Ship It!

Really. Ship it.

- Verification did its job — trust the pipeline, not one more gut-check

Watch it.

- Dashboards on. Alerts on. Treat new features as unfamiliar code until production says otherwise.

Let AI watch too.

- Anomalies, logs, remediation — acceleration doesn't stop at deployment

Then do it again — that's iterative improvement.

- Production is signal. Feed it back and adapt.
- Update the loop so it tightens every time.



Future Considerations / Parting Thoughts

01. Quality & Architectural Integrity

- **Maintaining Quality and Architectural integrity** over long spans of time (avoid the new 6 month cliff)
- Architects / teams must proactively nurture underlying technical concerns that might otherwise be missed

02. Team Dynamics

- **Use team workflows and cadences** to scale from individual contributor to team collaboration

03. Scaling

- **System & Application Scaling**
- Make sure growing your customer base won't topple the application

04. Enterprise Grade Development

- Additional technical care and compliance typically required
- Collaboration across teams, products, and organizations still requires many humans in the loop

Thank You!

Q&A

We'd love to hear your thoughts, feedback, and questions!

